

```
/**
```

- GUIDE REACT QUERY - BONNES PRATIQUES
  - 
  - Ce fichier explique les concepts clés de React Query et comment l'utiliser efficacement.
- ```
*/
```

```
// ===== 1. CONCEPTS DE BASE =====
```

```
/**
```

- React Query est une bibliothèque pour gérer l'état serveur dans React.
  - 
  - Elle gère automatiquement:
    - Le cache des données
    - Les états de chargement (loading)
    - Les erreurs
    - La synchronisation des données
    - Le refetching automatique
    - Les updates optimistes
- ```
*/
```

```
// ===== 2. QUERIES (LECTURE) =====
```

```
/**
```

- `useQuery` - Pour récupérer des données (GET)
  - 
  - Exemple basique:
- ```
*/  
import { useQuery } from "@tanstack/react-query"
```

```
function CategoriesList() {  
  const { data, isLoading, error, isError } = useQuery({  
    queryKey: ['categories'], // Identifiant unique  
    queryFn: fetchCategories, // Fonction qui retourne une Promise  
    staleTime: 5 * 60 * 1000, // Données considérées fraîches pendant 5 min  
    refetchOnWindowFocus: false, // Ne pas refetch au focus  
  })
```

```
  // Gérer les différents états  
  if (isLoading) return <div>Chargement...</div>  
  if (isError) return <div>Erreur: {error.message}</div>  
  
  return (  
    <ul>  
      {data?.map(cat => <li key={cat.id}>{cat.name}</li>)}  
    </ul>  
  )
```

```
    </ul>
  )
}
```

```
// ===== 3. MUTATIONS (ÉCRITURE) =====
```

```
/**
```

- useMutation - Pour modifier des données (POST/PUT/DELETE)
- 
- Exemple basique:

```
*/
```

```
import { useMutation, useQueryClient } from "@tanstack/react-query"
```

```
function CreateForm() {
```

```
  const queryClient = useQueryClient()
```

```
  const mutation = useMutation({
    mutationFn: createCategory,

    // Après succès, invalide le cache pour refetch
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['categories'] })
    },
  })

  const handleSubmit = (data) => {
    mutation.mutate(data)
  }

  return (
    <button
      onClick={() => handleSubmit({ name: "Tech" })}
      disabled={mutation.isPending}
    >
      {mutation.isPending ? "Création..." : "Créer"}
    </button>
  )
}
```

```
}
```

```
// ===== 4. OPTIMISTIC UPDATES =====
```

```
/**
```

- Optimistic Updates - Mettre à jour l'UI avant la réponse du serveur

- 
- Améliore l'UX en rendant l'interface instantanément réactive.

\*/

```
function OptimisticExample() {
  const queryClient = useQueryClient()
```

```
  const mutation = useMutation({
    mutationFn: createCategory,
```

```
    // AVANT l'envoi de la requête
    onMutate: async (newCategory) => {
      // 1. Annule les requêtes en cours (évite les conflits)
      await queryClient.cancelQueries({ queryKey: ['categories'] })

      // 2. Sauvegarde l'état actuel (pour rollback si erreur)
      const previousCategories = queryClient.getQueryData(['categories'])

      // 3. Met à jour optimistiquement le cache
      queryClient.setQueryData(['categories'], (old = []) => [...old,
        newCategory])

      // 4. Retourne le contexte (pour rollback)
      return { previousCategories }
    },

    // Si ERREUR, rollback
    onError: (err, newCat, context) => {
      queryClient.setQueryData(['categories'], context.previousCategories)
    },

    // Après SUCCESS ou ERROR, refetch les vraies données
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ['categories'] })
    },
```

```
  })
```

```
  return <button onClick={() => mutation.mutate({ name: "Tech" })}>Créer
}

// ===== 5. QUERY KEYS =====
```

/\*\*

- Query Keys - Identifiants uniques pour chaque requête
- 
- Bonnes pratiques:

- ◦ Utiliser des arrays
  - ◦ Plus c'est spécifique, mieux c'est
  - ◦ Organiser en hiérarchie
- \*/

// ✖ Mauvais

```
useQuery({ queryKey: 'categories', ... })
```

// ☑ Bon

```
useQuery({ queryKey: ['categories'], ... })
```

// ☑ Très bon - Hiérarchie

```
const categoryKeys = {
  all: ['categories'], // Toutes les requêtes de catégories
  lists: () => [...categoryKeys.all, 'list'], // Liste
  detail: (id) => [...categoryKeys.all, 'detail', id], // Détail
}
```

// Utilisation

```
useQuery({ queryKey: categoryKeys.lists(), ... })
useQuery({ queryKey: categoryKeys.detail('123'), ... })
```

// Invalidation ciblée

```
queryClient.invalidateQueries({ queryKey: categoryKeys.all }) // Invalide toutes les catégories
queryClient.invalidateQueries({ queryKey: categoryKeys.lists() }) // Invalide juste les listes
```

// ===== 6. CUSTOM HOOKS =====

/\*\*

- Custom Hooks - Encapsuler la logique React Query
  - 
  - Avantages:
    - ◦ Réutilisabilité
    - ◦ Meilleure organisation
    - ◦ Types TypeScript centralisés
- \*/

// ☑ Créer des hooks custom

```
export function useCategories() {
  return useQuery({
    queryKey: categoryKeys.lists(),
    queryFn: fetchCategories,
  })
}
```

```
export function useCreateCategory() {
  const queryClient = useQueryClient()
  return useMutation({
```

```

mutationFn: createCategory,
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: categoryKeys.all })
},
})
}

```

// Utilisation simple dans les composants

```

function MyComponent() {
  const { data: categories } = useCategories()
  const createMutation = useCreateCategory()

```

```

    return <div>...</div>

```

```

}

```

// ===== 7. GESTION DES ERREURS =====

/\*\*

- Gérer les erreurs de manière centralisée ou individuelle
- \*/

// Option 1: Dans le QueryClient (global)

```

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      onError: (error) => {
        console.error('Erreur query:', error)
        toast.error('Erreur de chargement')
      },
    },
    mutations: {
      onError: (error) => {
        console.error('Erreur mutation:', error)
        toast.error('Erreur lors de la sauvegarde')
      },
    },
  },
})

```

// Option 2: Par requête (spécifique)

```

const { data } = useQuery({
  queryKey: ['categories'],
  queryFn: fetchCategories,
  onError: (error) => {

```

```
toast.error(Impossible de charger les catégories: ${error.message})
},
})
```

// Option 3: Dans le composant

```
function MyComponent() {
  const { data, error, isError } = useCategories()
```

```
  if (isError) {
    return <ErrorAlert message={error.message} />
  }

  return <div>{/* ... */</div>
```

```
}
```

// ===== 8. PAGINATION =====

/\*\*

- Gérer la pagination avec keepPreviousData

\*/

```
function PaginatedCategories() {
  const [page, setPage] = React.useState(1)

  const { data, isPreviousData } = useQuery({
    queryKey: ['categories', { page }],
    queryFn: () => fetchCategories({ page }),
    keepPreviousData: true, // Garde les données précédentes pendant le chargement
  })
```

```
  return (
```

```
    {data?.categories.map(cat =>
      {cat.name}
    )}
```

```
    <button
      onClick={() => setPage(p => p - 1)}
      disabled={page === 1}
    >
      Précédent
```

```
    <button
      onClick={() => setPage(p => p + 1)}
      disabled={isPreviousData || !data?.hasMore}
    >
```

Suivant

```
)  
}
```

```
// ===== 9. PREFETCHING =====
```

```
/**
```

- Prefetch - Charger les données à l'avance

```
*/  
function CategoryItem({ categoryId }) {  
  const queryClient = useQueryClient()  
  
  const handleMouseEnter = () => {  
    // Prefetch les détails au survol  
    queryClient.prefetchQuery({  
      queryKey: categoryKeys.detail(categoryId),  
      queryFn: () => fetchCategory(categoryId),  
    })  
  }  
  
  return  
  
  ...  
  
}
```

```
// ===== 10. DEPENDENT QUERIES =====
```

```
/**
```

- Requêtes dépendantes - Une requête dépend du résultat d'une autre

```
*/  
function UserProfile({ userId }) {  
  // Charge l'utilisateur  
  const { data: user } = useQuery({  
    queryKey: ['user', userId],  
    queryFn: () => fetchUser(userId),  
  })  
  
  // Charge les posts SEULEMENT si on a l'utilisateur  
  const { data: posts } = useQuery({  
    queryKey: ['posts', user?.id],  
    queryFn: () => fetchUserPosts(user.id),  
    enabled: !!user, // Active uniquement si user existe  
  })  
}
```

```
return
```

```
...
```

```
}
```

```
// ===== 11. INFINITE QUERIES =====
```

```
/**
```

- Infinite Queries - Pour le scroll infini

```
*/
```

```
import { useInfiniteQuery } from "@tanstack/react-query"
```

```
function InfiniteCategories() {  
  const {  
    data,  
    fetchNextPage,  
    hasNextPage,  
    isFetchingNextPage,  
  } = useInfiniteQuery({  
    queryKey: ['categories', 'infinite'],  
    queryFn: ({ pageParam = 1 }) => fetchCategories({ page: pageParam }),  
    getNextPageParam: (lastPage, pages) => lastPage.nextPage ?? undefined,  
  })  
}
```

```
return (  
  <div>  
    {data?.pages.map((page, i) => (  
      <React.Fragment key={i}>  
        {page.categories.map(cat => (  
          <div key={cat.id}>{cat.name}</div>  
        ))}  
      </React.Fragment>  
    ))}  
    <button  
      onClick={() => fetchNextPage()}  
      disabled={!hasNextPage || isFetchingNextPage}  
    >  
      {isFetchingNextPage ? 'Chargement...' : 'Charger plus'}  
    </button>  
  </div>  
)
```

```
}
```

```
// ===== 12. BONNES PRATIQUES RÉSUMÉES =====
```



/\*\*

- ☒ À FAIRE:
- 
- 1. Toujours utiliser des arrays pour les query keys
- 2. Créer des custom hooks réutilisables
- 3. Organiser les query keys en hiérarchie
- 4. Utiliser optimistic updates pour meilleure UX
- 5. Invalider le cache après mutations
- 6. Gérer les états loading/error dans l'UI
- 7. Configurer staleTime selon vos besoins
- 8. Utiliser TypeScript pour les types
- 9. Tester vos hooks avec React Query Testing Library
- 10. Utiliser DevTools en développement

- ☒ À ÉVITER:
- 
- 1. Ne pas utiliser React Query pour l'état local (useState suffit)
- 2. Ne pas faire de fetch dans useEffect (utiliser useQuery)
- 3. Ne pas oublier d'invalider le cache après mutations
- 4. Ne pas dupliquer la logique (créer des hooks custom)
- 5. Ne pas ignorer les erreurs
- 6. Ne pas retry les mutations automatiquement
- 7. Ne pas mettre de données sensibles dans query keys

\*/

// ===== 13. EXEMPLE COMPLET =====

/\*\*

- Exemple complet d'un CRUD avec React Query
- \*/

// --- API Layer ---

```
export async function fetchCategories() {
  const res = await fetch('/api/categories')
  if (!res.ok) throw new Error('Erreur fetch')
  return res.json()
}

export async function createCategory(data) {
  const res = await fetch('/api/categories', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!res.ok) throw new Error('Erreur création')
```

```

return res.json()
}

// --- Hooks Layer ---
const categoryKeys = {
  all: ['categories'],
  lists: () => [...categoryKeys.all, 'list'],
}

export function useCategories() {
  return useQuery({
    queryKey: categoryKeys.lists(),
    queryFn: fetchCategories,
  })
}

export function useCreateCategory() {
  const queryClient = useQueryClient()
  return useMutation({
    mutationFn: createCategory,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: categoryKeys.all })
      toast.success('Catégorie créée !')
    },
    onError: (error) => {
      toast.error(error.message)
    },
  })
}

// --- Component Layer ---
function CategoriesManager() {
  const { data: categories, isLoading } = useCategories()
  const createMutation = useCreateCategory()

```

```

    const handleCreate = (data) => {
      createMutation.mutate(data)
    }

    if (isLoading) return <Spinner />

    return (
      <div>
        <CategoriesList categories={categories} />
        <CreateForm onSubmit={handleCreate} isLoading=
{createMutation.isPending} />
      </div>
    )
  }
}

```

```
</div>  
)
```

```
}
```